

SAKARYA UNIVERSITY

Faculty of Computer and Information Sciences

SWE 204 - CONCEPTS OF PROGRAMMING LANGUAGES

Homework 2 Report

Prepared by:

Elif Yırtıcı | Nurgül Dinçer

Instructor: Dr.Öğr.Üyesi Beyza Eken

Spring Semester 2025 – 2026

Presentation Date: 21 May 2026

Table of Contents

1. Introduction
2. Matrix Reading and Exception Handling
3. Implementations
4. Benchmark 1 — Data Structure Comparison
5. Benchmark 2 — Direct Computation vs Function Call
6. Observations and Analysis
7. Conclusion

1. Introduction

In this homework, we implemented matrix multiplication in C++, Java, and Python to analyze how different memory allocation strategies, data structures, and function calls affect performance. We also implemented exception handling for invalid matrix inputs and performed benchmarking experiments with increasing matrix sizes.

2. Matrix Reading and Exception Handling

The program reads two matrices from a single input file. Each matrix starts with its dimensions followed by rows of integer values.

During the reading process, the following input errors are checked:

- Missing elements in a row
- Extra elements in a row
- Non-numeric values
- Incompatible matrix dimensions for multiplication ($m \neq p$)

If any of these errors are detected, the program prints an appropriate error message and terminates execution for that file. This ensures robustness and reliability of the matrix processing system.

3. Implementations

Matrix multiplication was implemented using different data structures in each language to observe how internal memory handling affects performance.

3.1 C++ Implementations

- **Dynamic Array using new/delete** : In this implementation , memory is created on the heap using `new` and freed using `delete`. The matrix is kept in one continuous block of memory. Because the data is stored next to each other, the CPU cache very well. This makes moving through rows and columns faster. There are no extra pointer jumps, so large matrix operations run very quickly.
- **Vector (`vector<vector<int>>`)** : When you use `vector<vector<T>>`, each row is stored in a different place in memory. This makes the memory fragmented. Because the rows are not next to each other, the CPU cache is used less efficiently. As a result, large matrix operations run slower than when using one continuous array.

3.2 Java Implementations

- **2D Array (`int[ ][ ]`)** : In Java, a multi-dimensional array is actually an array of arrays. The main array keeps references (pointers) to each row. Each row stores `int` values next to each other in memory, but different rows are placed in different parts of the heap. Because of this, there is one extra pointer step. This makes the CPU cache a little less efficient when moving from one row to another during multiplication.
- **`ArrayList<ArrayList<Integer>>`** : This structure is slow for two main reasons: extra pointer steps and autoboxing. `ArrayList` stores objects, not primitive values. So each `int` must be stored as an `Integer` object. To read one value, the program goes through many steps: outer list → inner list → `Integer` object → real value. This creates fragmented memory, uses more space, and causes many cache misses.

3.3 Python Implementation

- **List of Lists**: In Python, a matrix is a list of lists that store references to integer objects. Integers are full objects, so the data is spread in memory with many pointer steps. This gives poor cache use and extra overhead in matrix multiplication loops.

4. Benchmark 1 — Data Structure Comparison

Language	Implementation	Size	Avg. Time (ms)
C++	Dynamic Array	50x80	0.448
C++	Vector	50x80	1.163
Java	Array	50x80	0.146
Java	ArrayList	50x80	0.329
Python	List	50x80	8.154

Language	Implementation	Size	Avg. Time (ms)
C++	Dynamic Array	120x150	3.184
C++	Vector	120x150	8.569
Java	Array	120x150	1.126
Java	ArrayList	120x150	2.887
Python	List	120x150	60.709

Language	Implementation	Size	Avg. Time (ms)
C++	Dynamic Array	300x400	84.943
C++	Vector	300x400	223.980
Java	Array	300x400	37.865
Java	ArrayList	300x400	102.255
Python	List	300x400	1590.461

Language	Implementation	Size	Avg. Time (ms)
C++	Dynamic Array	600x800	701.959
C++	Vector	600x800	1760.835
Java	Array	600x800	450.174
Java	ArrayList	600x800	850.873
Python	List	600x800	13776.539

Language	Implementation	Size	Avg. Time (ms)
C++	Dynamic Array	900x700	1421.939
C++	Vector	900x700	3510.436
Java	Array	900x700	1288.816
Java	ArrayList	900x700	1508.335
Python	List	900x700	24868.539

Observation:

In our observation, array structures performed best in Java. Dynamic arrays in C++ is one of the closest and quickest solutions to this. Vectors and ArrayLists introduce additional overhead due to internal memory management and resizing mechanisms. Python lists, being highly abstracted, showed slower performance compared to lower-level implementations.

5. Benchmark 2 — Direct Computation vs Function Call

Language	Implementation	Size	Avg. Time (ms)
C++	Direct	50x80	1.163
C++	Function Call	50x80	1.335
Java	Direct	50x80	0.329
Java	Function Call	50x80	0.331
Python	Direct	50x80	8.154
Python	Function Call	50x80	11.688

Language	Implementation	Size	Avg. Time (ms)
C++	Direct	120x150	8.569
C++	Function Call	120x150	9.445
Java	Direct	120x150	2.887
Java	Function Call	120x150	2.595
Python	Direct	120x150	60.709
Python	Function Call	120x150	87.876

Language	Implementation	Size	Avg. Time (ms)
C++	Direct	300x400	223.980
C++	Function Call	300x400	249.424
Java	Direct	300x400	102.255
Java	Function Call	300x400	99.121
Python	Direct	300x400	1590.461
Python	Function Call	300x400	2265.602

Language	Implementation	Size	Avg. Time (ms)
C++	Direct	600x800	1760.835
C++	Function Call	600x800	1933.934
Java	Direct	600x800	850.873
Java	Function Call	600x800	771.595
Python	Direct	600x800	13776.520
Python	Function Call	600x800	17998.434

Language	Implementation	Size	Avg. Time (ms)
C++	Direct	900x700	3510.436
C++	Function Call	900x700	3750.973
Java	Direct	900x700	1508.335
Java	Function Call	900x700	1385.321
Python	Direct	900x700	24868.539
Python	Function Call	900x700	34590.798

Observation:

While direct multiplication is observed to be faster than multiplication via a function in C++ and Python, in Java the situation changes depending on the matrix size. In a 50x80 matrix, direct multiplication is faster than multiplication via a function, but as the matrix size increases, multiplication via a function call becomes faster.

6. Observations and Analysis

The experiments clearly demonstrate the impact of memory management and abstraction levels on performance.

- Java arrays achieved the best performance, while C++ dynamic arrays performed moderately well; however, vectors and ArrayList structures caused noticeable slowdowns.
- Python showed the slowest performance due to dynamic typing, interpreter overhead, and high-level abstractions.
- Data structures that allocate memory contiguously outperform those that rely on fragmented memory or dynamic resizing.
- Function calls introduce measurable overhead in intensive loops, depending on the language and compiler optimizations.

These results highlight how implementation choices significantly influence computational efficiency.

7. Conclusion

This homework showed that memory management, abstraction level, and implementation choices strongly affect matrix multiplication performance.

Java arrays achieved the best performance in several tests, while C++ also performed efficiently due to contiguous memory allocation and low-level memory control. Python produced the slowest results because of dynamic typing and interpreter overhead.

The experiments also demonstrated that contiguous memory structures perform better than fragmented structures, and that the impact of function calls depends on the programming language and compiler optimizations.